

后端第5题

Task 1

Q1

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopyImage {
    public static void main(String[] args) {

        FileInputStream in = null;    // 输入流（读原图）
        FileOutputStream out = null; // 输出流（写新图）

        try {
            // 1. 打开流：读 doro.jpg，写 doro_copy.jpg
            in = new FileInputStream("doro.jpg");
            out = new FileOutputStream("doro_copy.jpg");

            // 2. 准备一个“中转箱子”，一次装 1024 个字节
            byte[] buffer = new byte[1024];

            // 3. 循环读，读到 -1 表示文件读完了
            int len;    // 存“这次实际读到了多少个字节”
            while ((len = in.read(buffer)) != -1) {
                // 4. 把读到的 len 个字节写到新文件
                out.write(buffer, 0, len);
            }

            System.out.println("复制成功！");
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            // 5. 关闭流（先关输出流，再关输入流）
            try {
```

```

        if (out != null) out.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    try {
        if (in != null) in.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
}

```

- 分析一下做题时的思路与犯的一些错误

1. 代码第八到九行，在开始的时候只是简单的定义了in和out没写= null只知道数字需要先定义值，不知道字节需要先定义，导致代码报错，问ds老师知道要先赋值为null，避免空指针错误
2. 不知道当读到文件末尾，read 会返回 -1 (-1 是"没东西可读了"的信号)，我当时甚至准备找方法数一下字节数
3. 关闭顺序：题目问"输入输出流的开关顺序"——先关输出流，再关输入流。为什么？因为要先确保数据"写完、保存好"（关输出流会刷新缓冲、把数据真正写进文件），再关输入。反过来可能丢数据

Q2

```

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class SortNames {
    public static void main(String[] args) {

        // 用"三层包裹"创建读文件的流
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(
                new FileInputStream("name.txt"),
                StandardCharsets.UTF_8))) {

```

```

List<String> names = new ArrayList<>();
String line;
while ((line = reader.readLine()) != null) {
    // trim 去掉首尾空格/换行
    String name = line.trim();
    // 如果是空行, 跳过
    if (name.isEmpty()) {
        continue;
    }
    names.add(name);
}

// 排序
Collections.sort(names);

// 用“三层包裹”创建写文件的流
try (BufferedWriter writer = new BufferedWriter(
    new OutputStreamWriter(
        new FileOutputStream("name_sorted.txt"),
        StandardCharsets.UTF_8))) {

    for (String name : names) {
        writer.write(name);    // 写名字
        writer.newLine();      // 换行
    }
}

System.out.println("排序完成!");

} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

- 分析思路与出的错误

1. 为什么不建议手写换行符?因为不同操作系统换行符不同 (Windows 是 `\r\n`, Linux 是 `\n`) `newLine()` 会自动用当前系统正确的换行符, 跨平台不会出错
2. 第 11~14 行: 三层包裹

- (读) `new FileInputStream("name.txt")`: 最里层, 字节流, 打开文件读字节
 - `new InputStreamReader(..., StandardCharsets.UTF_8)`: 中间层, 转换流, 把字节按 UTF-8 转成字符
 - `new BufferedReader(...)`: 最外层, 缓冲流, 提供 `readLine()` 一次读一行
3. 为什么这样写能自动关闭? 题目说"最外层关闭时, 内部包裹的流依次自动关闭"。所以用 `try (BufferedReader reader = ...)` 这种 `try-with-resources` 写法, 结束时只关最外层, 里面的会自动跟着关
4. `Name.txt`的位置我也弄了挺长时间, 我把文件下在`src`目录下运行, 代码报错说找不到这个文件, 查了一下发现要放在根目录下

Task 2

```
import java.io.*;

public class SerializeDemo {
    public static void main(String[] args) {

        // 1. 创建一个学生对象
        Student student = new Student(1, "doro", 2, "114514");

        // 2. 序列化: 把对象写入 student.dat 文件
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("student.dat"))) {
            oos.writeObject(student);
            System.out.println("序列化成功!");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 3. 反序列化: 从文件读出来, 赋给新对象 doro
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("student.dat"))) {
            Student doro = (Student) ois.readObject();
            System.out.println("反序列化成功!");
            System.out.println(doro);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

笔记与思路

1. 序列化 = 把一个对象（在内存里）转成字节数据，存到文件里

反序列化 = 反过来，从文件里读出字节，还原成对象
2. 为什么要序列化？对象在内存里是临时的，程序一关就没了。序列化能把对象"永久保存"到文件，或通过网络传输
3. Serializable：一个标记接口（里面没有任何方法，只是个"标记"）作用：告诉 Java"这个类的对象可以被序列化" 不加这个接口，序列化时会报错 `NotSerializableException`（不可序列化异常）
4. 关于流的开关时机：题目提示"输入输出流操作同一个文件，注意开关时机"。这里我用两个独立的 try 块——先序列化（写完自动关），再反序列化（读完自动关）。这样不会同时打开读写同一个文件，避免冲突

Task 3

- 什么是 Maven？Maven 是一个"项目管理工具"，帮你做三件事：
- 管理依赖：你要用别人写好的库（比如 fastjson），不用手动下载 jar 包，在配置文件里写一句，Maven 自动帮你下载
- 自动构建：一键编译、打包、测试你的项目
- 统一项目结构：规定了标准的目录结构
- 什么是 jar 包？jar 包 = 一堆 .class 文件（编译好的 Java 代码）打包压缩成的文件，类似"压缩包"
- 别人写好的工具（如 fastjson），会打包成 jar 给我们用 import 那些类，其实就是在用 jar 里的类
- Maven 和 jar 包什么关系？Maven 帮你管理 jar 包。以前要手动下载 jar、手动放进项目；现在在 pom.xml 里写一句依赖，Maven 自动去"仓库"下载对应的 jar
- 为什么要用 Maven？不用手动找 jar、下载、配置，省时省力 自动处理 jar 之间的依赖关系（A 依赖 B，你只写 A，Maven 会连带下载 B）团队协作时，大家用同一个 pom.xml，环境一致
- 下载 maven 的大致流程：下载 zip → 解压到 D:\maven → 配 MAVEN_HOME 和 Path → mvn -version 验证
- 我的 maven 版本为 3.10.0

仓库类型	在哪	一句话
本地仓库	你自己电脑上	你私人的"存货柜"
远程仓库	网上（公网/内网）	网上的"大超市"
中央仓库	Maven 官方的	全球最大的"总仓库"

- 配置方法：

找到 Maven 的配置文件：D:\maven\apache-maven-3.10.0\conf\settings.xml 用记事本打开。
找到 这一行 改成自己的路径

- 私服是什么？私服（Private Repository）= 公司/团队内部搭建的私有仓库

参数	全称	作用	例子
groupId	组织/公司标识	你是哪个组织的	<code>com.example</code>
artifactId	项目/模块名	这个项目叫什么	<code>express-system</code>
version	版本号	第几个版本	<code>1.0-SNAPSHOT</code>
三个参数			

命令	作用	记忆
<code>mvn clean</code>	清理（删掉 target 目录）	clean = 打扫
<code>mvn compile</code>	编译源代码	compile = 编译
<code>mvn test</code>	运行测试	test = 测试
<code>mvn package</code>	打包成 jar/war	package = 打包
<code>mvn install</code>	打包并装到本地仓库	install = 安装

- 我总结的易错点

1. 环境变量配了没生效 → 一定要重开命令行窗口
2. 路径有中文/空格 → 本地仓库、Maven 路径都不要有中文和空格。
3. IDEA 没关联自己的 Maven → 要去 Settings 里改 Maven home path
4. 改了 pom.xml 不刷新 → 记得点 Maven 面板的刷新按钮。java 放错位置 → 必须放 src/main/java 下
5. 依赖坐标写错 → groupId/artifactId/version 任何一个字母错就下载失败，去 mvnrepository 核对
6. jar 下载慢或失败 → 考虑换镜像（阿里云镜像），在 settings.xml 里配

Task 4

```
import com.alibaba.fastjson2.JSON;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

public class Server {
    private static final int PORT = 8000;
    // 模拟快递柜数据: key = 快递单号_手机号, value = 取件码
    private static final Map<String, String> expressMap = new HashMap<>();

    public static void main(String[] args) throws IOException {
        initializeExpressData();
        HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
        server.createContext("/query", new QueryHandler());
        server.start();
        System.out.println("Server started on port " + PORT);
    }

    private static void initializeExpressData() {
        expressMap.put("SF123456789_13005433678", "1234");
        expressMap.put("JD987654321_19805433168", "5678"); // TODO1: 补全
        expressMap.put("YT456789123_13905479698", "9012"); // TODO1: 补全
        expressMap.put("ZT789123456_18505433664", "3456"); // TODO1: 补全
    }

    static class QueryHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if (!"POST".equals(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(405, -1);
            }
        }
    }
}
```

```
        exchange.close();
        return;
    }

    try {
        String body = new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
        QueryRequest request = JSON.parseObject(body, QueryRequest.class);
        String trackingNumber = request.getTrackingNumber();
        String phone = request.getPhone();

        // TODO2: 校验单号和手机号不能为空
        if (trackingNumber == null || trackingNumber.isEmpty()
            || phone == null || phone.isEmpty()) {
            throw new IllegalArgumentException("单号和手机号不能为空");
        }

        // TODO3: 拼 key 查询
        String key = trackingNumber + "_" + phone;
        String pickCode = expressMap.get(key);

        // TODO4: 构造响应
        QueryResponse response;
        if (pickCode != null) {
            response = new QueryResponse(pickCode, "success");
        } else {
            response = new QueryResponse(null, "手机号不正确");
        }

        // TODO5: 序列化并发送
        String responseJson = JSON.toJSONString(response);
        byte[] bytes = responseJson.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(200, bytes.length);
        OutputStream os = exchange.getResponseBody();
        os.write(bytes);
        os.close();
    } catch (Exception e) {
        // TODO6: 返回400
        try {
            String errorJson = "{ \"pick_code\":null, \"msg\": \"请求格式错误\" }";
            byte[] bytes = errorJson.getBytes(StandardCharsets.UTF_8);
```

- 初始化数据: `expressMap` 是 `HashMap`, 键是 "单号_手机号", 值是取件码 `put(键, 值)`: 存进去。这是模拟的"快递柜数据库"
- 创建服务器: `HttpServer.create(...)`: 创建一个 HTTP 服务器, 监听 8000 端口
`createContext("/query", ...)`: 设置路由——当有人访问 `/query` 这个路径时, 交给 `QueryHandler` 处理 `server.start()`: 启动服务器 (开始监听)
- 处理: `exchange.getRequestMethod()`: 获取请求方式 (GET 还是 POST)
`!"POST".equals(...)`: 如果不是 POST, 就返回 405, 然后 `return` 结束 注意写法
`"POST".equals(...)` (把 "POST" 放前面) 是为了防止空指针
- 发送响应: `JSON.toJSONString(response)`: 把响应对象转成 JSON 字符串 `getBytes(...)`: 把字符串转成字节数组 (要发的是字节) `sendResponseHeaders(200, bytes.length)`: 发送响应头, 状态码 200, 内容长度 `getResponseBody()`: 拿到响应的输出流 `os.write(bytes)`: 写响应体。 `os.close()`: 关闭 `import com.alibaba.fastjson2.JSON;`

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    while (true) {
        try {
            System.out.print("请输入快递单号（输入exit退出）：");
            String trackingNumber = scanner.nextLine().trim();
            // TODO1: 输入exit退出
            if ("exit".equals(trackingNumber)) {
                break;
            }

            System.out.print("请输入手机号：");
            String phone = scanner.nextLine().trim();

            // TODO2: 构造JSON请求体
            QueryRequest request = new QueryRequest(trackingNumber, phone);
            String requestJson = JSON.toJSONString(request);

            // TODO3: 创建连接并发请求
            URL url = new URL(SERVER_URL);
            HttpURLConnection connection = (HttpURLConnection) url.openConnection();
            connection.setRequestMethod("POST");
            connection.setRequestProperty("Content-Type", "application/json");
            connection.setDoOutput(true);

            OutputStream os = connection.getOutputStream();
            os.write(requestJson.getBytes(StandardCharsets.UTF_8));
            os.close();

            // 读取响应
            if (connection.getResponseCode() == 200) {
                // TODO4: 读取并解析
                String body = readBody(connection.getInputStream());
                QueryResponse response = JSON.parseObject(body, QueryResponse.class);
                if (response.getPick_code() != null) {
                    System.out.println("取件码：" + response.getPick_code());
                } else {
                    System.out.println(response.getMsg());
                }
            } else {

```

```

        System.out.println("查询失败，状态码：" + connection.getResponseCo
    }
    connection.disconnect();

    } catch (IOException e) {
        System.out.println("请求发生错误：" + e.getMessage());
    }
}
scanner.close();
System.out.println("客户端已退出");
}

private static String readBody(InputStream is) throws IOException {
    StringBuilder sb = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(is, Stan
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line);
        }
    }
    return sb.toString();
}
}

```

client大致是与上面的服务端相通的：先读输入，构造JSON，发送请求，读响应